

CENTRO UNIVERSITARIO UNINASSAU
CURSO DE CIENCIA DA COMPUTACAO

NovaLang: Projeto de Compilador Basico

Integrantes:

Lucas Azevedo da Rocha Cortez
Marcos Tulio Cicero Bezerra Magalhaes

Disciplina: Compiladores

Professor: Anderlan Henrique Batista Siqueira

Maceio, 2026

Sumário

1	Introducao	2
2	Gramatica da Linguagem	2
3	Analise Lexica	3
3.1	Tokens Reconhecidos	3
3.2	Exemplo Comentado	3
4	Conclusao	3
4.1	Uso de Inteligencia Artificial	4
4.2	Aprendizados	4

1 Introducao

Este trabalho descreve o desenvolvimento de **NovaLang**, uma linguagem de programacao educacional implementada em Python. O objetivo e demonstrar, de forma pratica e didatica, os principios fundamentais da construcao de compiladores, percorrendo as tres fases classicas: analise lexica, analise sintatica e interpretacao [1].

A linguagem foi projetada para suportar:

- Operacoes matematicas (+, -, ×, ÷) com precedencia correta e agrupamento por (), [] e {};
- Declaracao e chamada de funcoes com parametros e valor de retorno;
- Estrutura condicional if/else;
- Laco de repeticao while;
- Exibicao na tela via print;
- Captura de entrada do usuario via input.

O desenvolvimento contou com auxilio de inteligencia artificial como ferramenta de apoio, detalhada na Secao 4.

2 Gramatica da Linguagem

A gramatica é definida em notacao **BNF** (*Backus-Naur Form*) [2].

Listing 1: Gramatica BNF de NovaLang

```
1 <programa>      ::= <instrucao>*
2 <instrucao>     ::= <atrib> | <if> | <while> | <func> | <print> | <
   return>
3 <atrib>         ::= IDENT '=' <expressao>
4 <expressao>     ::= <termo> (('+' | '-') <termo>)*
5                 | <expressao> OP_COMP <expressao>
6 <termo>         ::= <fator> (('*' | '/') <fator>)*
7 <fator>         ::= NUMERO | IDENT
8                 | '(' <expressao> ')'
9                 | '[' <expressao> ']'
10                | '{' <expressao> '}'
11                | <chamada_funcao> | <input>
12 <if>            ::= 'if' <expr> '{' <instrucao>* '}' ('else' '{' ...
   '}'')?
13 <while>         ::= 'while' <expr> '{' <instrucao>* '}'
14 <func>          ::= 'func' IDENT '(' params? ')' '{' <instrucao>* '}'
15 <print>         ::= 'print' '(' <expr> ')'
16 <input>         ::= 'input' '(' STRING ')'
17 <return>        ::= 'return' <expr>
```

A hierarquia `expressao` → `termo` → `fator` implementa a precedencia matematica: × e ÷ resolvem antes de + e −; grupos entre delimitadores tem precedencia maxima.

3 Analise Lexica

A **analise lexica** e a primeira fase do compilador. Ela percorre o código-fonte e agrupa caracteres em unidades chamadas **tokens** — as menores unidades com significado.

3.1 Tokens Reconhecidos

A tabela abaixo lista todas as categorias de tokens reconhecidas pelo analisador lexico de NovaLang.

Token	Padrao (regex)	Exemplo
NUMERO	<code>\d+(\.\d+)?</code>	42, 3.14
STRING	<code>"[^\"]*"</code>	"Ola"
IF	<code>\bif\b</code>	if
ELSE	<code>\belse\b</code>	else
WHILE	<code>\bwhile\b</code>	while
FUNC	<code>\bfunc\b</code>	func
IDENT	<code>[a-zA-Z_]\w*</code>	x, soma
OP	<code>[+\-*/]</code>	+, *
OP_COMP	<code>== != <= >= < ></code>	>=
ATRIB	<code>=</code>	=

3.2 Exemplo Comentado

Entrada: `resultado = (10 + [5 * 2]) - {3}`

Listing 2: Tokens gerados pelo lexer

<code>('IDENT', 'resultado')</code>	<code><- nome de variavel</code>
<code>('ATRIB', '=')</code>	<code><- atribuicao (nao e comparacao)</code>
<code>('LPAREN', '(')</code>	<code><- abre grupo com parenteses</code>
<code>('NUMERO', 10)</code>	<code><- literal int (texto -> int)</code>
<code>('OP', '+')</code>	<code><- adicao</code>
<code>('LBRACKET', '[')</code>	<code><- abre grupo com colchetes</code>
<code>('NUMERO', 5)</code>	
<code>('OP', '*')</code>	<code><- multiplicacao (precedencia maior)</code>
<code>('NUMERO', 2)</code>	
<code>('RBRACKET', ']')</code>	
<code>('RPAREN', ')')</code>	
<code>('OP', '-')</code>	<code><- subtracao</code>
<code>('LBRACE', '{')</code>	<code><- abre grupo com chaves</code>
<code>('NUMERO', 3)</code>	
<code>('RBRACE', '}')</code>	

Calculo: $10 + (5 \times 2) - 3 = 17$

4 Conclusao

O projeto demonstrou como as tres fases de um compilador colaboram de forma modular. A separacao em `lexer.py`, `parser.py` e `interpreter.py` tornou o código extensível e facilitou a depuracao isolada de cada etapa.

4.1 Uso de Inteligencia Artificial

O modelo Claude (Anthropic) foi utilizado como ferramenta de apoio em quatro situações:

1. **Estrutura do lexer:** Sugestão do uso de `re.finditer` com grupos nomeados. Avaliado, compreendido e adotado.
2. **Precedencia matematica:** $2 + 3 * 4$ retornava 20. A IA sugeriu a hierarquia *parsear_expressao* $\rightarrow$ *parsear_termo*. Implementado e validado.
3. **Escopo de funcoes:** Variaveis locais vazavam para o escopo global. Corrigido com copia e restauracao do dicionario de ambiente.
4. **Divisao por zero:** Sem mensagem amigavel. Corrigido com verificacao explicita antes da operacao.

4.2 Aprendizados

Compiladores são sistemas modulares com responsabilidades claras por camada. A IA acelerou o desenvolvimento, mas cada sugestão foi compreendida e validada antes de ser adotada. Como extensão futura: vetores, tipos explícitos e mensagens de erro com número de linha.

Referências

- [1] AHO, Alfred V. et al. *Compiladores: Principios, Tecnicas e Ferramentas*. 2. ed. São Paulo: Pearson, 2007.
- [2] LOUDEN, Kenneth C. *Compiler Construction: Principles and Practice*. Boston: PWS Publishing, 1997.